I actually think you've reached the most important point of the project.

Don't write any more PRDs. Start building the specification repository.

If you look at companies like **Stripe**, **Linear**, **Vercel**, or open-source projects like **Kubernetes**, they don't have one giant design document. They have hundreds of small documents, each responsible for one concept.

This is how I would proceed

Instead of this

```
docs/

World_Poetry_Explorer_Detailed_PRD_v1.md

WPExplorer_structure.md
```

I'd immediately reorganize into this:

```
docs/
│
├── README.md
│
├── 00-overview/
├── 01-philosophy/
├── 02-product/
├── 03-domain/
├── 04-ontology/
├── 05-information-architecture/
├── 06-pages/
├── 07-design-system/
├── 08-data-model/
├── 09-api/
├── 10-backend/
├── 11-editorial/
├── 12-roadmap/
└── adr/
```

Don't worry if most folders are empty initially.

Now here's the important part.

Never ask AI

> "Build my app."

Instead

Ask

> "Write docs/04-ontology/themes.md"

Once that's finished

Ask

> "Write docs/04-ontology/symbols.md"

Then

> "Write docs/08-data-model/poems.md"

etc.

This is exactly how I'd use AI.

The order matters

I'd spend almost **2–3 weeks** without writing a single line of code.

Seriously.

Because this project is **knowledge-heavy**.

Week 1

Understand the domain.

```
docs/

README

Vision

Mission

Philosophy

Human Experience
```

```
Knowledge Graph
```

Week 2

Understand literature.

```
Themes

Symbols

Poets

Schools

Languages

Civilizations
```

Week 3

Design database.

```
Poets table

Poems table

Verse table

Theme table

Symbol table

Relationship table

Indexes
```

Only then...

Week 4

Backend.

Week 5

Frontend.

Why?

Because coding is only about **20%** of this project.

The remaining 80% is defining the intellectual model.

I'd also create one file called

```
docs/brain.md
```

This becomes your notebook.

Whenever you get an idea

don't keep it in your head.

Write it there.

For example

```
Idea

Users should be able to compare
Ghalib's understanding of death
with Shakespeare.

Status

Not planned.

Priority

Medium.
```

Another

```
Idea

Each symbol should have its own page.
```

```
Example

Wine

History

Islamic usage

Persian usage

Christian usage

Greek usage

Related Poems

Related Poets
```

This becomes the incubator for future features.

Here's something I think will save you months of work

I would **not** start by documenting pages.

Instead, I would start by documenting **the domain**.

Most startups design screens first.

I would design **the knowledge graph first**.

For example, before deciding what the Poem page looks like, we should answer questions like:

- What is a **Poem**?
- What is a **Verse**?
- What is a **Theme**?
- What is a **Human Experience**?
- What is a **Symbol**?
- Can a symbol belong to multiple civilizations?
- Can a poem have multiple translations?
- Can a poem express contradictory themes?
- How do we distinguish "love" from "divine love" and "unrequited love"?

Once those concepts are rigorously defined, the database, APIs, search, recommendations, and UI almost design themselves.

If I were your Technical Product Architect

This is the exact sequence I'd follow over the next few weeks:

Phase	Deliverable	Outcome
1	README + Vision + Philosophy	Everyone understands the mission
2	Domain Model	Define poets, poems, verses, themes, symbols, concepts
3	Ontology	Build the taxonomy of human experiences and literary symbols
4	Database Schema	Translate the ontology into normalized tables
5	Knowledge Graph	Define relationships between all entities
6	API Contracts	Design stable interfaces before implementation
7	UI Specifications	Create screen-by-screen behavior and layouts
8	Implementation	Let AI generate the code in phases

One more recommendation

I think this project deserves its **own GitHub repository** from day one, even before you write the application code.

```
world-poetry-spec/     ← Documentation repository
world-poetry-api/      ← Backend (later)
world-poetry-web/      ← Frontend (later)
world-poetry-cms/      ← Editorial CMS (later)
```

This separation has two major benefits:

1. **The specification becomes the authoritative source of truth.** Any AI coding agent—or human developer—can consult it without mixing documentation with implementation.
2. **The specification becomes reusable.** Whether you build the web app, mobile apps, or future AI features, they all rely on the same documented domain model rather than

duplicating assumptions.

If we proceed this way, we're not just documenting an app—we're designing a durable platform that can evolve for years while keeping its intellectual foundation consistent.